

PRD: llmctl — a unified terminal dashboard for multi-provider LLM development

Status: Draft v1 **Owner:** Ayush Kushwah **Context:** B.Tech CSE capstone project (7–8 month timeline, mid-point demo required) and intended public open-source release

1. Problem Statement

What problem does this solve?

Developers using multiple LLM providers (Anthropic, OpenRouter, Ollama, and others) inside AI coding agents like Claude Code lose real, repeated time to environment friction: mismatched env var syntax between PowerShell and WSL, provider-specific auth schemes, wrong or stale model identifiers, and silently broken local services (Redis, WSL, Ollama). None of this is a "how do I call an LLM" problem — it is a workflow and diagnostics problem that today has no dedicated tool.

When a developer does switch providers mid-task (e.g., moving from a frontier model to a cheaper/local one to continue a debugging session), there is no way to carry the conversation context and tool-use history across that switch without manually copy-pasting, and no record afterward of which model said what. Naively replaying the full raw transcript to the new provider also re-bills the entire conversation as input tokens on every switch, with no visibility into that cost beforehand.

Who experiences this problem?

Individual developers and small teams on Windows who use WSL as their primary Linux dev environment and regularly work across 2+ LLM providers in the same coding session — a fast-growing group as multi-model workflows (frontier model for reasoning, cheap/local model for volume) become standard practice.

Why does this problem matter now?

The number of viable LLM providers and coding agents (Claude Code, Codex, OpenCode, local models via Ollama) is growing quickly, and cost-conscious developers are increasingly routing different tasks to different models. Windows/WSL remains a large and underserved segment of this workflow — most existing tooling in this space (LiteLLM, `llm`) is Linux/Mac-first and solves API routing, not the interactive terminal experience, the Windows-specific environment problems, or the cost/coherence problem of switching models mid-conversation.

2. Target User

Primary user: Individual developers on Windows using WSL, who use one or more AI

coding agents/CLIs (Claude Code, OpenRouter-backed tools, local Ollama models) as part of their daily workflow. Comfortable in the terminal; not necessarily deep systems experts.

Goals:

- Switch between LLM providers/models without re-configuring the shell every time
- Trust that the environment (WSL, Redis, auth, network) is actually working before starting a task, not after a failure
- Keep working across a long session even if a model runs out of budget, hits a rate limit, or simply isn't performing well — without losing the thread of the conversation, and without silently re-paying for the whole conversation on every switch

Frustrations:

- Env vars set correctly in PowerShell don't carry into WSL and vice versa
- Typo'd or outdated model identifiers cause failures deep inside a session, not up front
- No single place to check "is everything actually working right now"
- Switching providers mid-task means starting over, manually reconstructing context, or accepting a large, invisible token cost

Behaviors:

- Keeps a terminal window open for the entire work session (this is why a TUI, not a one-shot CLI, fits their workflow)
 - Already comfortable installing and configuring CLI tools, editing config files, managing API keys
-

3. Core Features (MVP)

Must-have

1. Provider profile management Store and manage config profiles (API keys, base URLs, default models) for Anthropic, OpenRouter, and Ollama in one encrypted local config file, added and edited from within the TUI.

2. One-action provider/model switching with correct env propagation Switch the active provider and model from inside the dashboard; the tool detects PowerShell vs. WSL/Bash and exports the correct environment variables for that shell automatically.

3. Distillation-based context handoff with attribution As a session progresses, the tool continuously extracts small, structured "notes" from the conversation (decisions made, facts established, code state, open questions) — not a running text summary, but discrete, tagged records stored individually. When the user switches provider/model mid-conversation, instead of replaying the full raw transcript, the tool sends the accumulated notes plus the last few raw turns to the new provider. Every note and every transcript

message is tagged with the provider/model that produced it, and the TUI shows a visible divider at each switch point. Before confirming a switch, the user sees an estimated token/cost comparison between a full replay and the distilled handoff.

4. Live diagnostics panel ("doctor") A persistent status view (not just an on-demand command) showing WSL state, Redis status, network reachability, and auth validity for each configured provider, refreshed continuously while the dashboard is open.

5. Session transcript and notes view, with export Scrollable, attributed conversation history within the TUI, plus a separate view of the extracted notes for the session (Obsidian-style — small linked records, not a wall of text). Both exportable as markdown/JSON.

Nice to have (post-MVP, explicitly deferred)

- **Retrieval-based session memory** (embedding the extracted notes and retrieving only the relevant subset at handoff time, instead of sending all notes) — intentionally excluded from this project's scope. See Section 8.
 - macOS / Linux / ARM builds (Windows/WSL is the v1 platform target)
 - Side-by-side response comparison across providers for the same prompt
 - Cost/token usage tracking per provider over time
 - Plugin system for community-contributed provider adapters
 - Config sync between multiple machines
-

4. Out of Scope

- Retrieval-based (embedding + semantic search) session memory — see Section 8 for why this is deferred and how the schema stays ready for it
 - Hosted/web dashboard or any server-side component — this is fully local, no accounts
 - Team or multi-user features of any kind
 - Any monetization, billing, or paid tier
 - macOS, Linux, and ARM builds (planned for later, not this project)
 - Telemetry, analytics, or crash reporting of any kind
 - A general-purpose LLM gateway/proxy (this is not a LiteLLM competitor — no request routing for third-party applications)
 - Support for providers beyond Anthropic, OpenRouter, and Ollama at launch
-

5. Success Metrics

Capstone / demo metrics (by mid-point review):

- Working demo: provider switch with distillation-based handoff, shown live, without errors
- Live demo shows the pre-switch cost comparison (full replay vs. distilled) with real numbers
- Diagnostics panel correctly detects at least 3 real failure states (broken WSL, expired key, unreachable Ollama) in a live demo

Public release metrics (post-launch, ongoing):

- GitHub stars / installs as a proxy for organic adoption in the target niche
 - Number of unique users completing a provider switch within their first session (activation)
 - Qualitative: unsolicited feedback or issues filed by real Windows/WSL users outside your own network
-

6. Technical Assumptions

- **Language/framework:** Go, using Bubble Tea for the TUI, Lip Gloss for styling, Bubbles for prebuilt components (spinners, tables, viewport for scrollable transcript)
 - **Platform (this project):** Windows with WSL2; PowerShell and WSL/Bash shell detection built in from the start
 - **Platform (future, out of scope now):** macOS (Intel + Apple Silicon), Linux (x64 + ARM64) — provider-adapter and storage architecture should not assume Windows-only, to avoid a rewrite later
 - **Config storage:** Local encrypted file (e.g., using `age` or equivalent) — no cloud storage
 - **Notes/session storage:** SQLite, with notes stored as structured rows, not flat text. See Section 8 for schema details — this is the key design decision that keeps a future retrieval upgrade additive rather than a rewrite
 - **Provider adapters:** Common interface (`validate()`, `listModels()`, `getEnvVars()`, `sendMessage()`) per provider, so Anthropic/OpenRouter/Ollama are three implementations of one interface, not three separate code paths
 - **Note extraction:** A decoupled pipeline step — a small/cheap model call (or the active model itself, off the critical path) turns recent conversation turns into structured notes. This step's output format stays fixed even if what happens to the notes afterward changes
 - **No backend/server component** — the binary is the whole product
 - **Integrations:** None beyond the three LLM provider APIs themselves; no Stripe, no OAuth/social login, no third-party analytics
-

7. Open Questions

1. Should note extraction run continuously in the background as the session progresses, or only be triggered at the moment of a provider switch? (Affects perceived latency vs. handoff speed.)
 2. Should the encrypted config store use a user-supplied passphrase, or rely on OS-level credential storage (Windows Credential Manager) to avoid asking for a password every session?
 3. What's the minimum viable set of "doctor" checks for the mid-point demo vs. the fuller set for launch?
 4. Should the tool assume the user already has Go toolchains / WSL configured, or does v1 need a guided first-run setup flow?
 5. How many raw recent turns should always be sent verbatim alongside the distilled notes (a fixed number, or based on remaining context budget)?
 6. For the public release: soft-launch quietly in Windows/WSL and Claude Code communities, or hold for a more polished v1 before any public visibility?
-

8. Extensibility note: designing for a future retrieval-based upgrade

Retrieval-based session memory (embedding each note and semantically retrieving only the relevant subset at handoff time, rather than sending all accumulated notes) is a stronger, more scalable version of the distillation approach — but it adds real engineering scope (embeddings, vector storage, ranking logic) that is intentionally **out of scope for this project**. It is documented here so the system is built in a way that adding it later is additive, not a rewrite.

What this project must get right for that migration to be smooth:

- **Notes are structured SQLite rows, not a growing text blob.** Minimum schema:

```
notes(  
  id, session_id, content, provider, model,  
  timestamp, tags, links_to, superseded_by  
)
```

A future retrieval system needs exactly this table plus an embedding vector per row — no reshaping of existing data.

- **Note extraction is a separate pipeline stage from "what happens to the notes after."** This project: extraction → concatenate all relevant notes → send. A future version: extraction → embed + retrieve top-k → send. Same extraction code, different consumer of its output.

- **Notes are tagged with topic/context metadata at creation time**, even though this project only uses tags for display/filtering, not for retrieval ranking. Capturing this now is far cheaper than backfilling it by re-processing old sessions later.
- **No shortcut of storing the "distilled summary" as a single paragraph.** Every distilled fact must be its own row. This is the one discipline that, if skipped for speed, would force real rework later.

This project should explicitly NOT build: embeddings, vector search, similarity ranking, or link-graph traversal. Those stay documented here as the natural next phase, should there be a reason to pick this up again after the capstone.